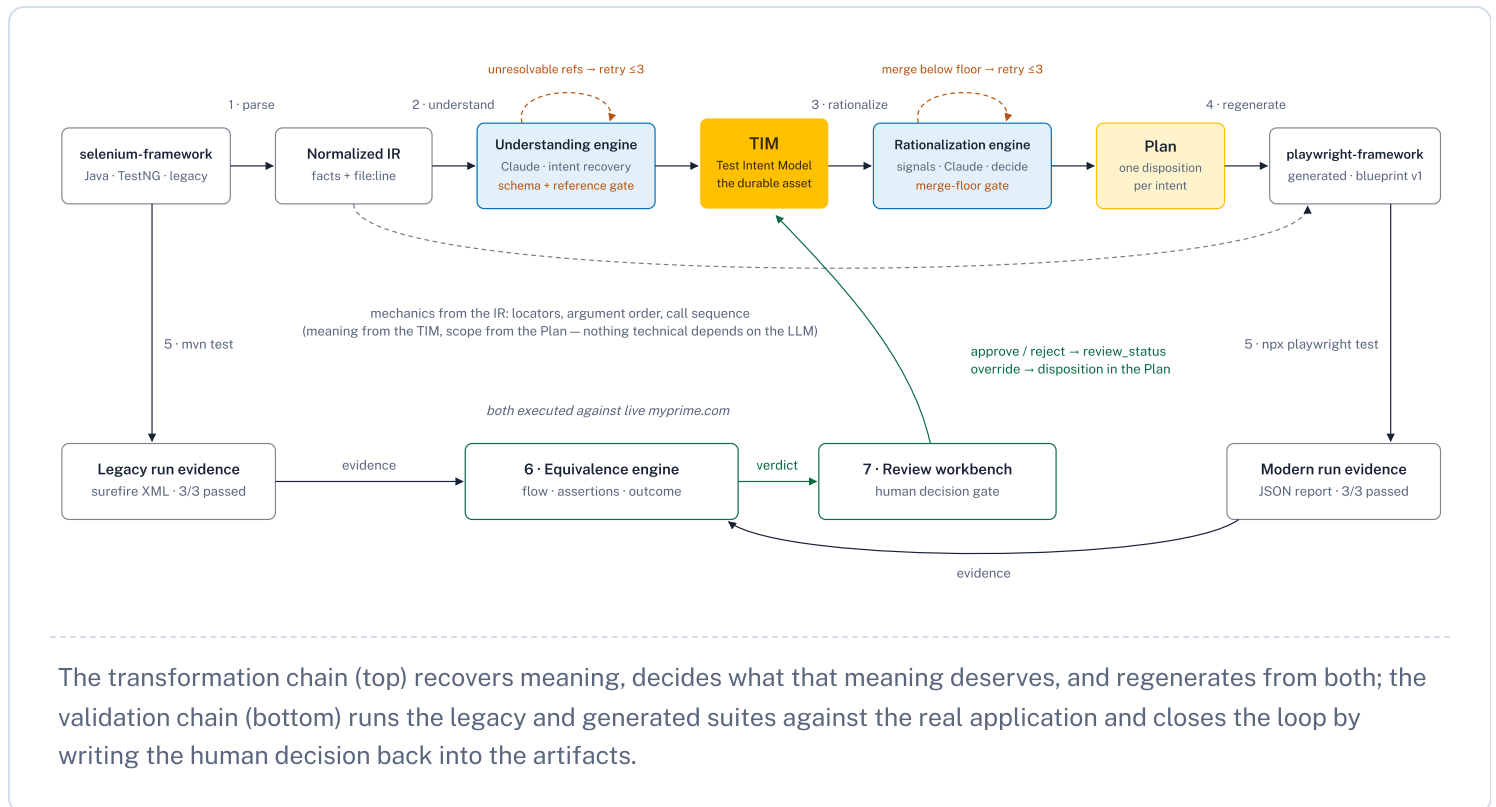


Test Intent Pipeline

The pipeline never translates Selenium code into Playwright code. It recovers **what each test means** into a framework-neutral Test Intent Model, decides **what each intent deserves** before anything is rebuilt, regenerates a new implementation from that meaning under enterprise conventions, and then **proves** both implementations verify the same business behavior — with a human owning every decision.

The closed loop

Seven stages. AI answers exactly two narrow questions — *what does this test mean?* and *what does it deserve?* — and each answer has to pass its own deterministic gate before it becomes an artifact anything else depends on.



Seven stages, one command each

The chips say who does the work. Parsing, generation, execution, and comparison are deterministic code. AI proposes meaning and dispositions behind validation gates; a human owns the final call on both.

1 Ingest

DETERMINISTIC

```
python pipeline.py ingest
```

A source adapter parses the Java framework — page objects, locators, action sequences, test methods, assertions, config, oracle data — into a **Normalized Technical IR** where every node carries its source file and line.

Real run: **4 page objects · 26 locators · 3 tests · 15 assertions** → `normalized_ir.json`

2 Understand

AI + VALIDATION

```
python pipeline.py understand
```

Claude recovers business intent from the IR as schema-conformant JSON — e.g. *“search for ‘atorvastatin’, select the matching suggestion, specify strength, quantity, frequency.”* Validators reject any reference that doesn’t resolve against the IR and feed the errors back for retry.

Real run: **3 TIM tests, avg confidence 0.88**; two ambiguous sanity checks honestly flagged at 0.6

3 Rationalize

AI + VALIDATION

```
python pipeline.py rationalize
```

Before anything is rebuilt, decide what each recovered intent *deserves*. Deterministic signals come first — pairwise redundancy, how much a test actually proves, what it costs to run, whether the platform can regenerate it at all. Claude then assigns a disposition and writes the business rationale, and every rationale must cite those signals by name.

Real run: TIM-001 and TIM-002 scored **0.925 redundant** → merged; TIM-003 scored **0.23** → migrated as its own test

4 Generate

DETERMINISTIC

```
python pipeline.py generate
```

Mechanics from the IR, meaning from the TIM, scope from the Plan: emits the whole Playwright framework under an enterprise blueprint — page objects, shared fixtures, data-driven oracles, and `TIM-001` traceability in every test title. Legacy timing quirks become named resilience patterns, not transliterated loops.

Real run: **25/25 methods regenerated** — 21 generic, 4 blueprint patterns, **0 TODO stubs**; two merged intents emitted as **one parameterized body**

5 Execute

DETERMINISTIC

```
python pipeline.py execute
```

Runs the legacy suite (`mvn test`) and the generated suite (`npm playwright test`) against the live application, normalizing both reports into per-test evidence: status, duration, failure detail.

Caught **2 real behavioral gaps** pre-approval: duplicate element ids, and a missing constructor load-gate

6 Compare

DETERMINISTIC

```
python pipeline.py compare
```

Explainable equivalence per test — never a bare pass/fail diff: is every intent step realized, is every business assertion regenerated against the same oracle, and did both implementations verify the same values live? An intent that was deliberately retired or deferred reads as `NOT_MIGRATED` , not as a gap.

Real run: flow **3/3** · assertions **7/7** · outcomes match → **3× EQUIVALENT**

7 Review

HUMAN GATE

```
streamlit run review_ui/app.py
```

The workbench opens on the portfolio: every intent's disposition, its rationale, and the signals behind it — with an override control that records the decision as human-made and recomputes the portfolio numbers. Then, per test: recovered intent with confidence and evidence, legacy vs generated code side by side, execution results, and the verdict's reasoning. Approve / Reject writes `review_status` back into the TIM.

Nothing AI-inferred becomes approved fact without a human decision — not the meaning of a test, and not its fate.

What an intent can deserve

A 1:1 port of every legacy test is the failure mode of modernization, not the goal. Estates carry redundancy, tests that assert nothing about the application, and coverage whose value no longer justifies its cost — so the disposition is a decision the platform makes explicitly, with evidence, before writing any code.

MIGRATE

Sound, distinct, worth its cost. Port it.

MERGE

The same test wearing different data. Fold the implementations together.

SPLIT

Proves several unrelated things; should become several tests.

REDESIGN

Valuable intent, wrong realization — proved through the UI when it isn't a UI concern.

RETIRE

Obsolete, or proves nothing about the application under test.

DEFER

Valuable but blocked. Must name what blocks it.

Why the AI can't hallucinate a migration

GUARDRAIL • REFERENTIAL

Every claim must resolve

Each TIM step cites a `binding_ref` into the IR and each expected value a reference into the oracle data (`data:drugs.LIPITOR_BRAND.detailTierHeading`). Unresolvable output is rejected, not accepted.

GUARDRAIL • EVIDENTIAL

Confidence + evidence on every value

Every step and assertion carries a confidence score and `file:line` evidence back to source. Anything under 0.7 is flagged for human review instead of passing silently.

GUARDRAIL • PORTFOLIO

A merge has to beat a number

An LLM will call two tests redundant because their names rhyme. A MERGE is rejected unless every pair in the group clears the computed redundancy floor — and every rationale must cite a signal that actually exists in the pack.

GUARDRAIL • BEHAVIORAL

Merging consolidates code, never coverage

A realized merge emits one parameterized body with a case per intent, each keeping its own expected values and its own TIM id. If the bodies differ anywhere that isn't a literal, the generator **refuses** the merge and emits them separately.

GUARDRAIL • PROVENANCE

The platform stamps, the human approves

Model, prompt version, and timestamp are stamped by platform code — never by the model. Approval and disposition overrides exist only as a reviewer's action in the workbench.

The run, in numbers

3

TIM tests recovered

0.88

avg intent confidence,
honestly scored

0.925

measured redundancy
behind the merge

2→1

implementations after
the merge

25/25

methods regenerated, no
stubs

3/3

passing live, both
frameworks

3x

verdict: EQUIVALENT

“We parse the legacy framework deterministically, ask AI two narrow questions — **what does this test mean?** and **what does it deserve?** — validate both answers against source evidence and computed signals, regenerate a modern framework from that meaning, run both against the real app, and hand a reviewer explainable proof they check the same behavior. **The intent model in the middle is framework-neutral** — the same TIM could generate Cypress or pytest tomorrow without re-reading a line of Selenium.”

```
selenium-framework → modernization-platform → playwright-framework · artifacts: ir / tim /  
rationalization / runs / reports
```